

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Implementierung und Matlab-Praxis 5:

M-Programmierung

Inhalte

5. M-Programmierung

- 5.1 Besonderheiten der M-Programmierung
- 5.2 Debugging
- 5.3 Hilfreiche Funktionen

5. M-Programmierung – 5.1 M-Programmierung

Eigenschaften des Matlab-Editor:

- Aufruf über Doppelklick auf eine m-Datei im Explorer oder Matlab oder
- über den Befehl

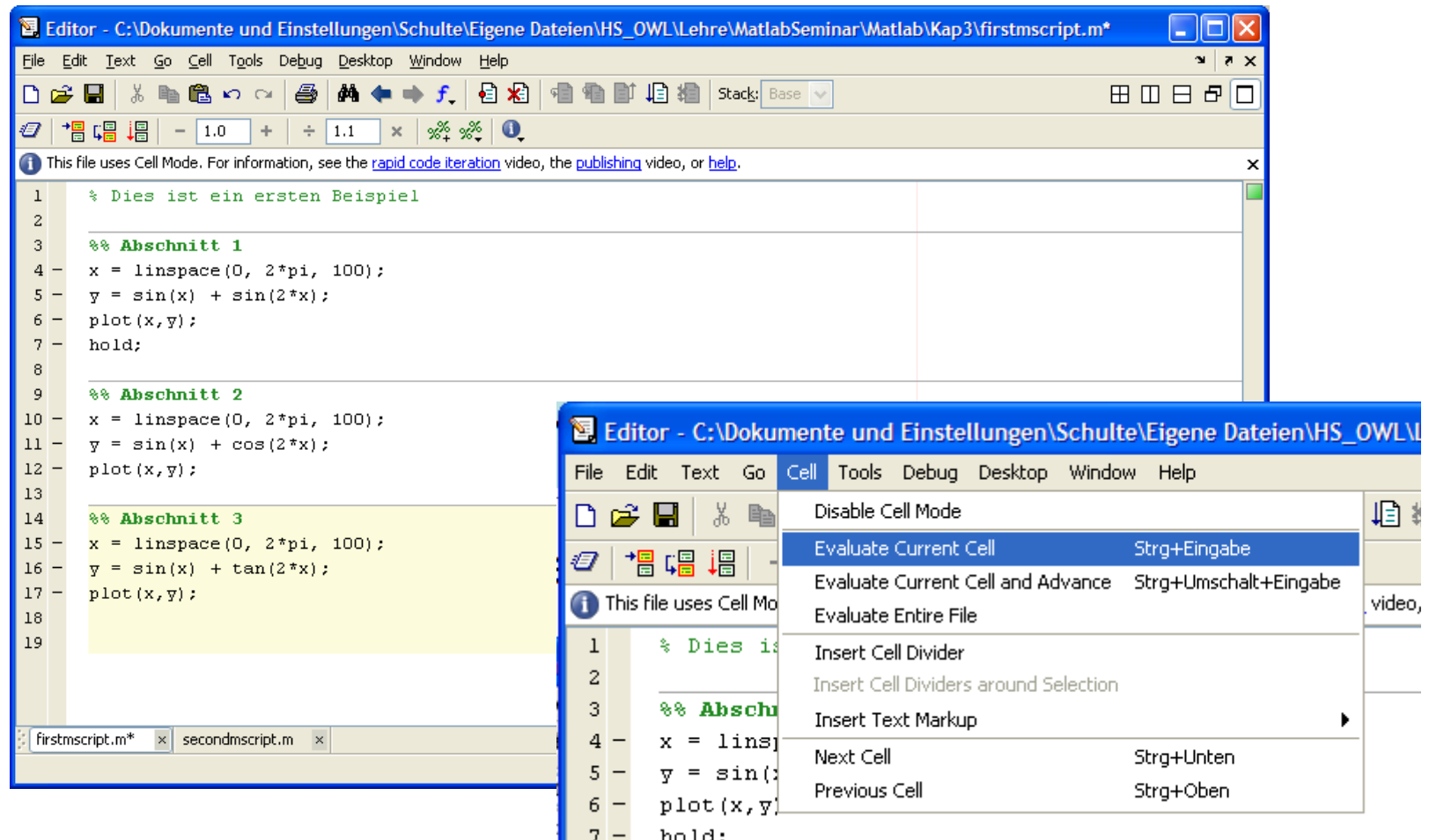
```
>> edit <dateiname>
```

- Spezielles *Highlighting* für die Matlab-Syntax
- Strukturierung und Einrückhilfen (‘on-the-fly’ oder *Control-I*)
- Debug-Funktionen
- Strukturiertes Arbeiten mit Zellen
- Veröffentlichungsfunktionen



5. M-Programmierung – 5.1 M-Programmierung

Strukturiertes Arbeiten durch Zellen (Cells):



5. M-Programmierung – 5.1 M-Programmierung

Try-Catch-Anweisung :

Fehlerbehandlung:

```
try
    <Anweisung>
catch
    <Anweisungen für den Fehlerfall>
end;
```

Beispiele:

```
a1 = [1, 2, 3];
a2 = [4, 5];
```

```
try
    A =[a1;a2];
catch
    disp('Ein Fehler ist aufgetreten');
end
```



5. M-Programmierung – 5.1 M-Programmierung

Grundlegende Regeln zur m-Programmierung:

- Benutzen Sie eindeutige Variablen- und Funktionsnamen und nutzen Sie die Datenstrukturen von Matlab sinnvoll aus.
- Realisieren Sie sinnvolle Fehlerbehandlungen:
 - Bauen Sie Überprüfungen von Funktionseingaben in Ihre Funktionen ein.
 - Nutzen Sie die `try...catch` Anweisung zur Fehlerbehandlung.
 - Realisieren Sie sinnvoll Fehlerreaktionen und Ausgaben von „Error-Messages“ und „Warnings“.
- Nutzen Sie nach Möglichkeit die vektorisierten Funktionen von Matlab anstelle von Schleifenprogrammierung
(→ Verbesserung der Rechengeschwindigkeit).
- Schreiben Sie sinnvoll Kommentare und versehen Sie insbesondere die Funktionen mit einem Kommentar im Kopf, der Syntax und Funktion eindeutig beschreibt.

5. M-Programmierung – 5.1 M-Programmierung

Eingabe-Anweisung:

Die input-Anweisung ermöglicht Benutzereingaben im Sinne einer Abfrage in einem Stript:

```
...  
alter = input('Geben Sie Ihre Alter an: ');  
...
```

Geben Sie Ihre Alter an: ...

```
...  
aelter30 = input('Sind Sie älter als 30 Jahre [y/n]: ','s');  
...
```

Sind Sie älter als 30 Jahre [y/n]:...

```
...  
aelter30 = input('Sind Sie älter als 30 Jahre? \nWenn ja dann...  
bitte y eingeben [y/n]: ','s');  
...
```

Sind Sie älter als 30 Jahre?
Wenn ja dann bitte y eingeben [y/n]: ...



5. M-Programmierung – 5.1 M-Programmierung

Ausgabe-Anweisung:

Einfach:

```
disp(<string>);
```

Beispiel:

```
disp(['Ihr Alter ist ',num2str(alter),' Jahre']);
```

Leistungsfähiger: `sprintf` oder `fprintf`

```
...  
fprintf('Ihr Alter ist %0.0i Jahre.\n',alter);  
...
```

```
Ihr Alter ist 22 Jahre.  
>>
```

Wichtig:

```
fprintf  
sprintf
```

→ File oder Bildschirmausgabe
→ Ausgabe in String



5. M-Programmierung – 5.1 M-Programmierung

Formatierungssyntax:

Angelehnt an C lautet die Syntax für `fprintf` und `sprintf`:

```
fprintf(<fileID>,<formatstring>,<A1>,<A2>,...)
```

<code>fileID</code>	→	1. File-ID aus der <code>fopen</code> -Anweisung 2. Wert 1 für Bildschirmausgabe (ist auch Default) 3. Wert 2 für Error-Ausgabe
---------------------	---	---

<code>formatstring</code>	→	Beliebiger String mit Text, Steuer- und Formatierungszeichen
---------------------------	---	---

<code>A1, A2</code>	→	Folge von Arrays die entsprechend der Formatierungszeichen umgesetzt werden.
---------------------	---	---

5. M-Programmierung – 5.1 M-Programmierung

Aufbau des `formatstring`:

1. Beliebiger Text wobei die Steuerzeichen dargestellt werden durch:

<code>' '</code>	→ Hochkommata durch Doppelschreibweise
<code>% %</code>	→ Prozentzeichen durch Doppelschreibweise
<code>\ \</code>	→ Backslash durch Doppelschreibweise

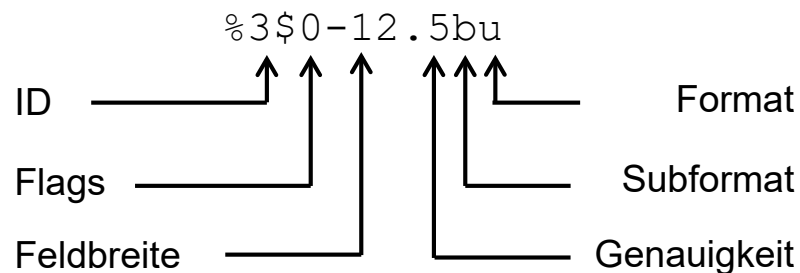
2. Steuerzeichen (Escape Characters):

<code>\b</code>	→ Backspace
<code>\f</code>	→ Form feed
<code>\n</code>	→ Neue Zeile
<code>\r</code>	→ Carriage return
<code>\t</code>	→ Horizontaler Tabulator
<code>\xN</code>	→ Hexadecimalzahl, N
<code>\N</code>	→ Octalzahl, N

5. M-Programmierung – 5.1 M-Programmierung

3. Formatierungszeichen (Conversion Characters):

Das %-Zeichen führt einen Formatierungsstring an:



Erläuterung:

ID	→ Position in der Array-Liste
Flags	→ Siehe Tabelle oder Hilfe
Format	→ Siehe Tabelle

5. M-Programmierung – 5.1 M-Programmierung

Flags:

'_'	→ linksbündig
'+'	→ Vorzeichen darstellen
' '	→ Leerzeichen vor Wert
'0'	→ mit '0' auffüllen

Format u. Subformat:

Integer:	%d or %i	→ Dezimal
	%ld or %li	→ 64-bit Dezimal
UInteger:	%u	→ Dezimal
	%o	→ Oktal
	%X	→ Hexadezimal
Floating:	%f	→ Normal
	%e	→ Exponentialschreibweise
	%g	→ Kompakte Exponentialschr.
Strings:	%c	→ Buchstabe (Character)
	%s	→ String

5. M-Programmierung – 5.1 M-Programmierung

Weiteres Beispiel:

```
Desc      = 'Zwischenkreisspannung';  
Unit      = 'V';  
Comment   = 'zu hoch'  
Value     = 1000;  
  
fprintf(1,'Der Wert der %2$s beträgt %4$+5.2f%2$s.\nDieser Wert von  
    . . . %4$+5.2f%2$s ist %3$s.\n',Desc,Unit,Comment,Value);
```

Der Wert der V beträgt +1000.00V.
Dieser Wert von +1000.00V ist zu hoch.

Anzeigen eines Hyperlinks:

```
site = 'http://www.mathworks.com';  
title = 'The MathWorks Web Site';  
  
fprintf('<a href = "%s">%s</a>\n', site, title);
```

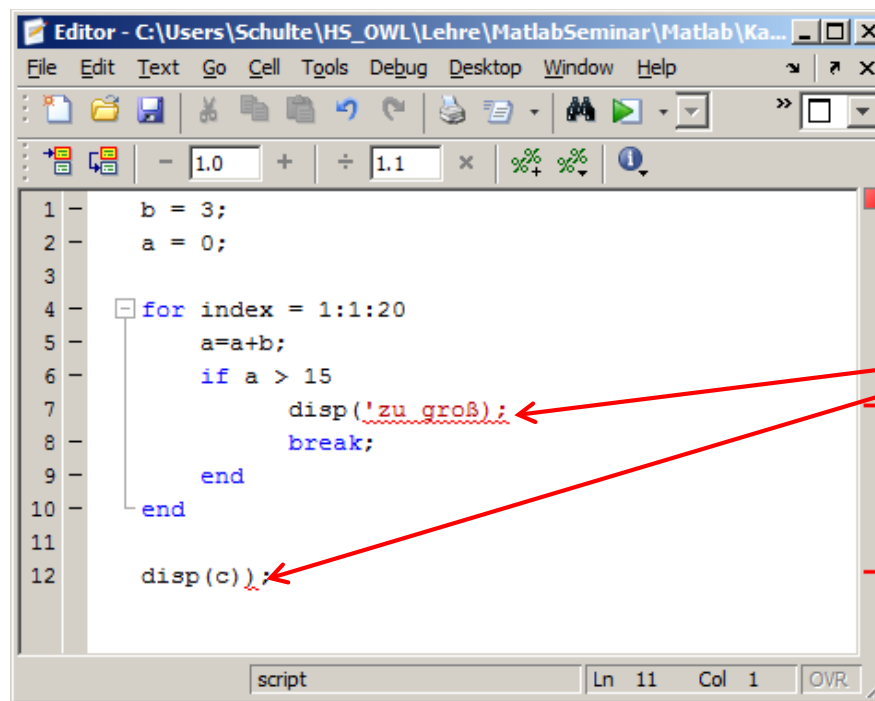
[The MathWorks Web Site](http://www.mathworks.com)

5. M-Programmierung – 5.2 Debugging

Debugging:

Der Matlab-Editor bietet verschiedene Möglichkeiten des Debugging.

1. Syntax-Prüfung



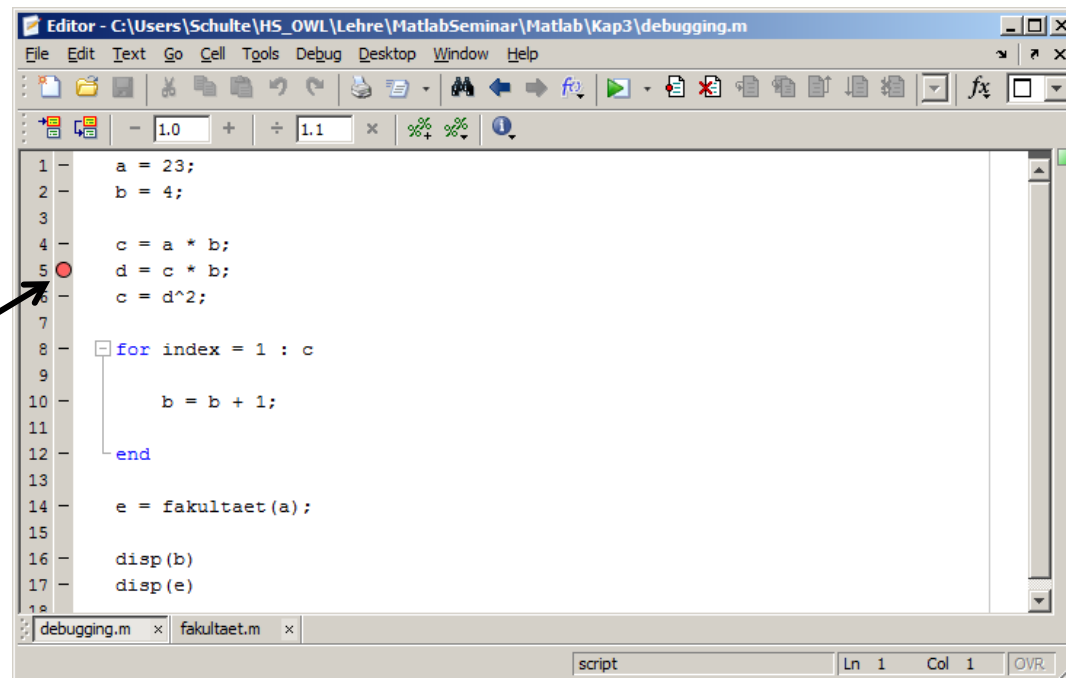
Anzeigen von
Syntaxfehlern
und Warnungen

5. M-Programmierung – 5.2 Debugging

2. Debugging von Laufzeitfehlern:

Ein fehlerhafter Abbruch oder ein unerwünschtes Ergebnis lässt sich häufig nur schwer debuggen. Anstelle des „üblichen“ Einfügens von Zwischenausgaben bietet der Matlab-Editor die Möglichkeit in Skripten und Funktionen sog. Break-Points einzufügen oder eine schrittweise Bearbeitung durchzuführen.

Breakpoint



5. M-Programmierung – 5.2 Debugging

Nach dem Start des Skripts stoppt die Bearbeitung am Breakpoint (die letzte Operation ist die vor dem Breakpoint) und lässt sich bis zu nächsten Breakpoint oder schrittweise (d. h. zeilenweise) fortsetzen.

Breakpoint setze, bzw. löschen

Schrittweise Bearbeitung

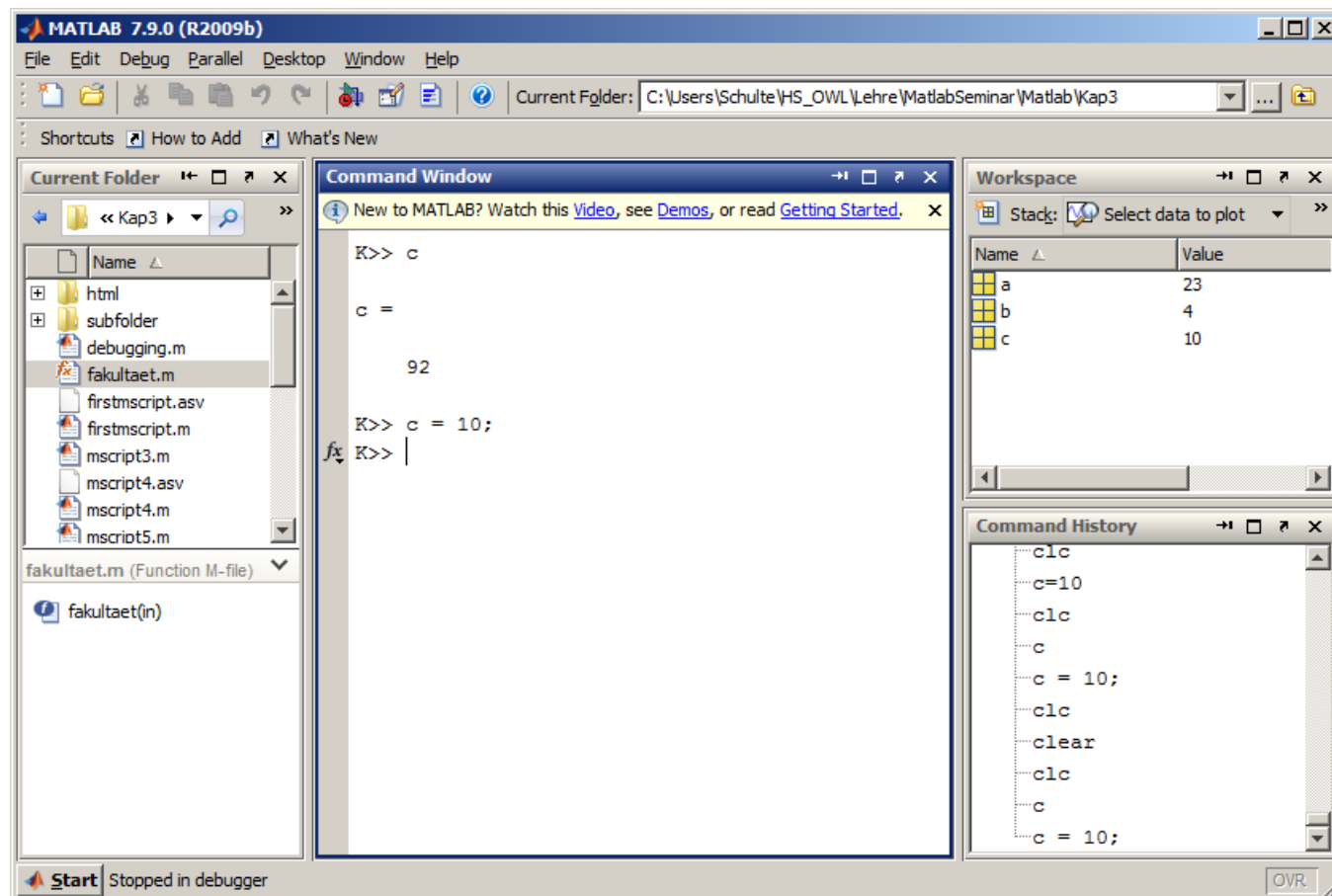
Bearbeitung bis zum nächsten Breakpoint

Indikator für die aktuelle Anweisung

```
1 - a = 23;  
2 - b = 4;  
3 -  
4 - c = a * b;  
5 - d = c * b;  
6 - c = d^2;  
7 -  
8 - for index = 1 : c  
9 -  
10 -     b = b + 1;  
11 -  
12 - end  
13 -  
14 - e = fakultaet(a);  
15 -  
16 - disp(b)  
17 - disp(e)  
18 -
```

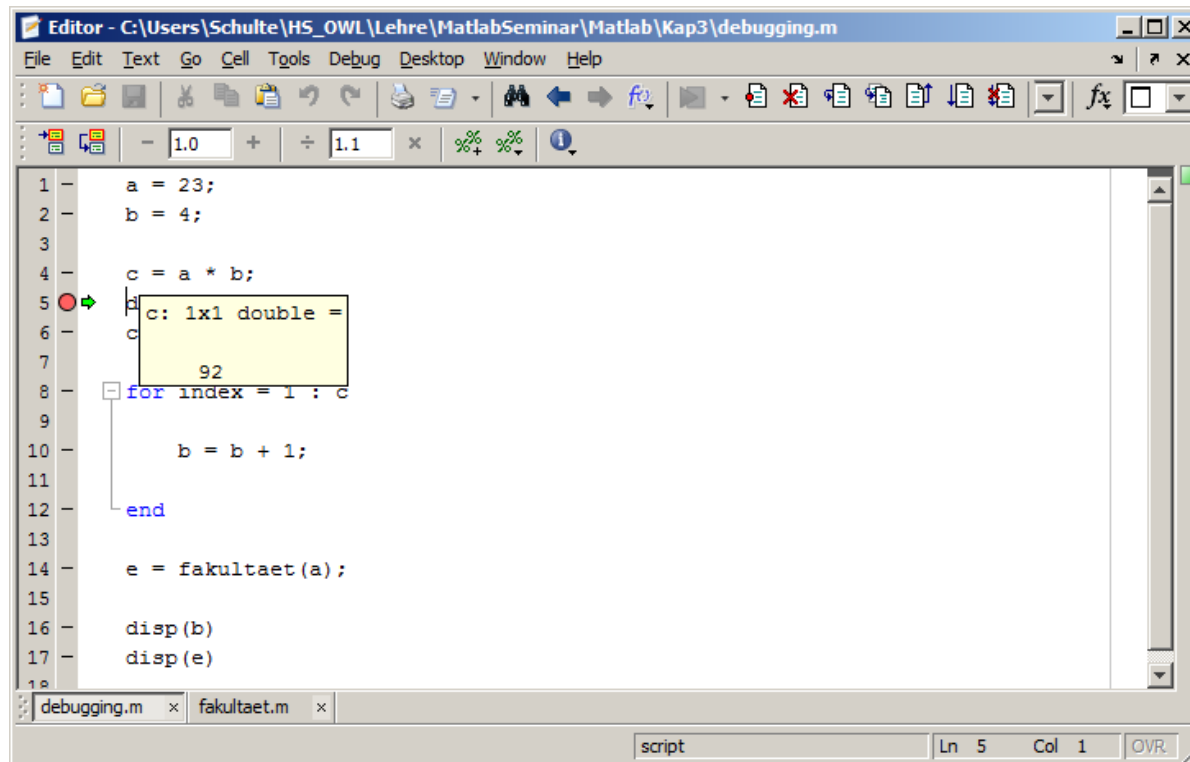

5. M-Programmierung – 5.2 Debugging

Im Debugging-Mode verändert sich das Kommando-Prompt (K>>) und es ist möglich mit den aktuellen Zwischenwerten zu arbeiten, sie sogar zu verändern. Auch der Workspace-Browser zeigt den aktuellen Zwischenstand.



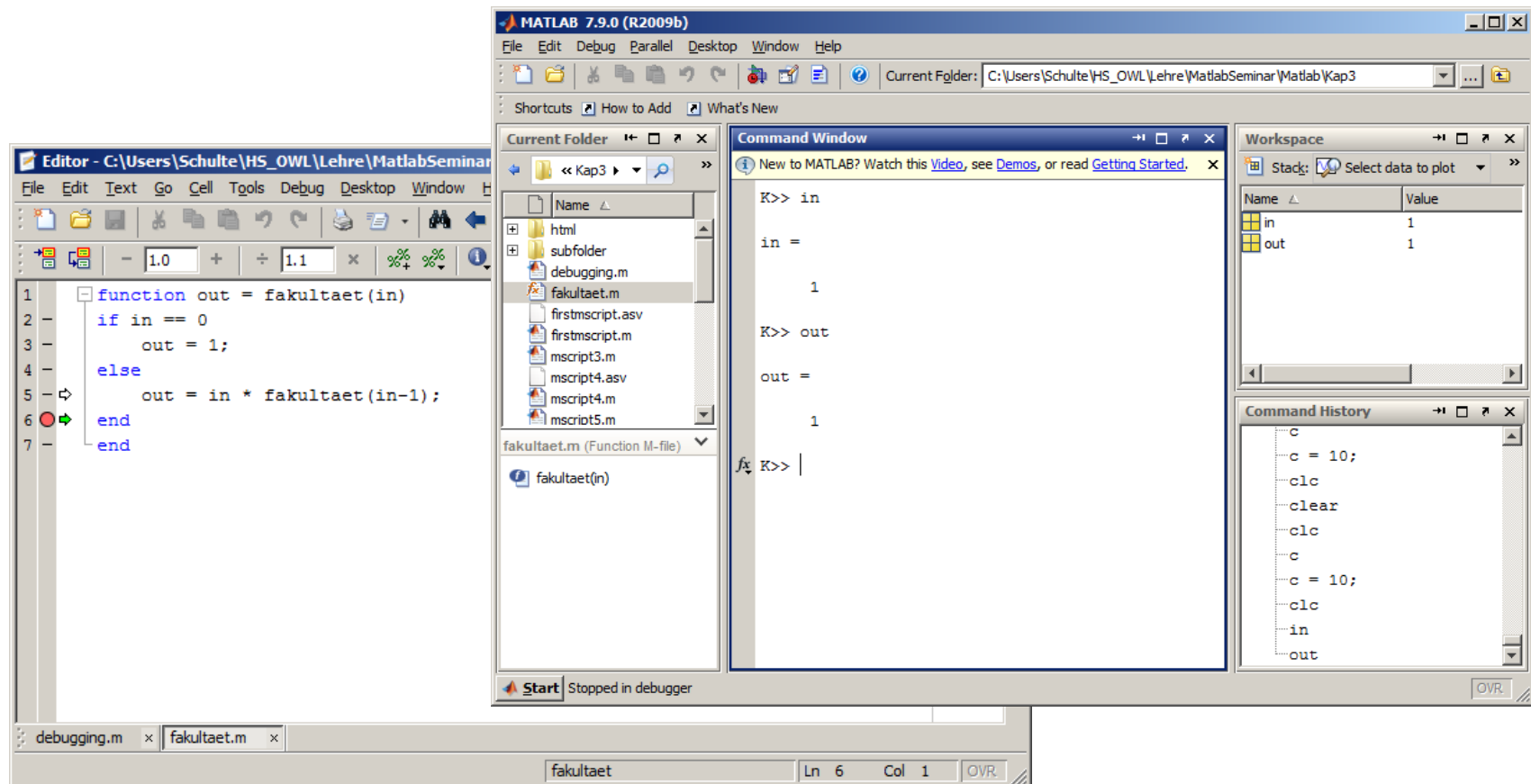
5. M-Programmierung – 5.2 Debugging

Ein Variable wird im Debugging-Mode auch angezeigt, wenn man den Mauszeiger daneben platziert.



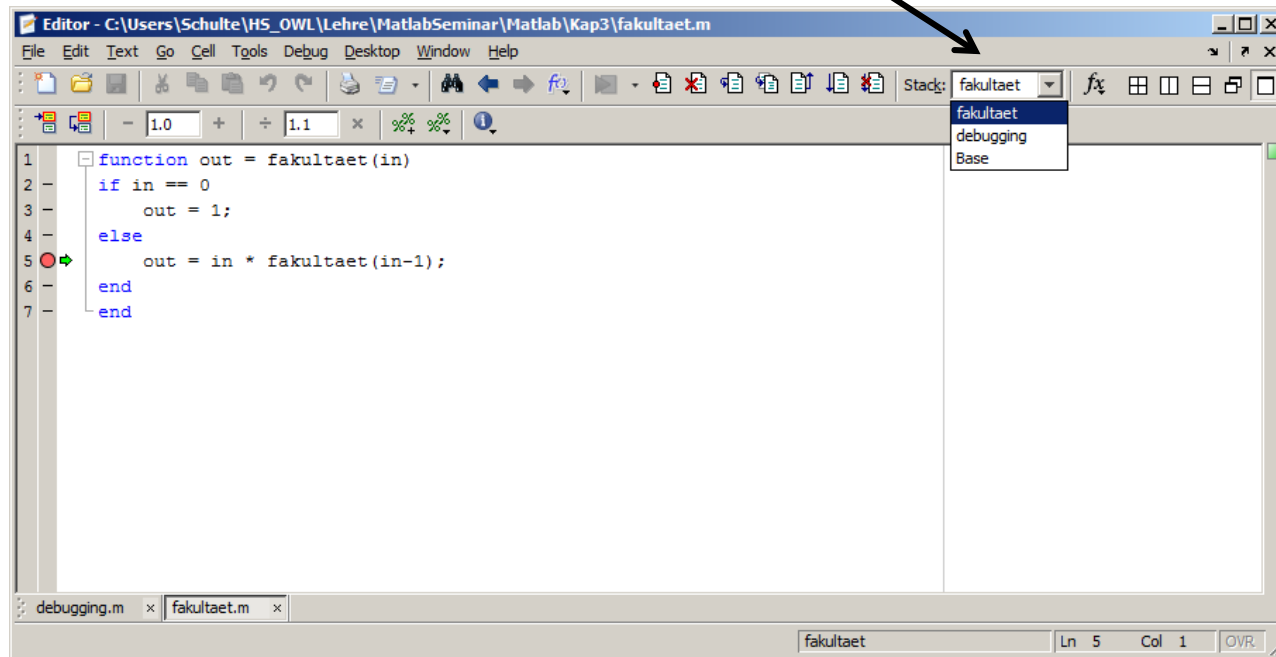
5. M-Programmierung – 5.2 Debugging

Es ist auch möglich Breakpoints und schrittweise Abarbeitung in Funktionen zu nutzen. Dabei besteht im Debugging-Mode direkter Zugriff auf den Workspace der Funktion (s. u.), mit den gleichen Manipulationsmöglichkeiten wie bei Skripten.



5. M-Programmierung – 5.2 Debugging

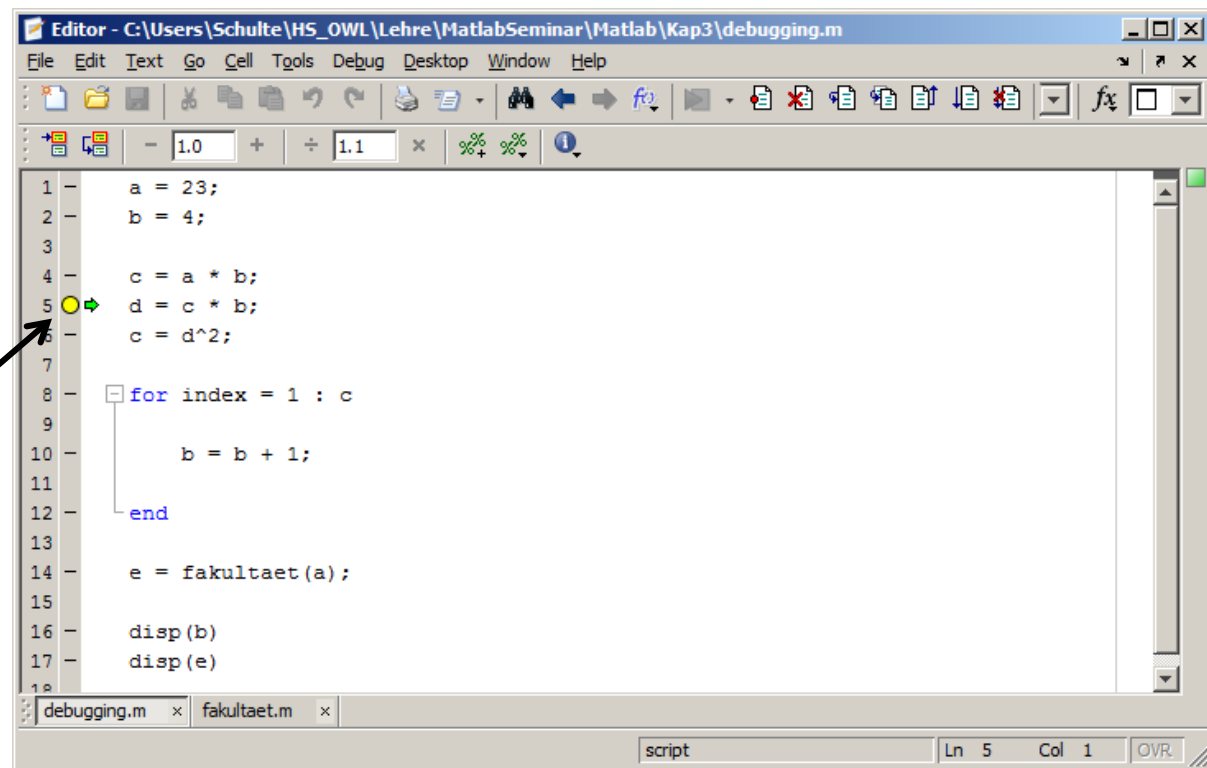
Die „Stack“-Anzeige zeigt im Debugging-Mode den aktuellen Workspace und ermöglicht auch eine Umschaltung für die Anzeige um Matlab-Fenster.



5. M-Programmierung – 5.2 Debugging

Neben „normalen“ Breakpoints gibt es bedingte Breakpoint (können über das Menü → Debug oder rechte Maustaste gesetzt werden). Es kann auch ein Stop bei Fehlern oder Warnungen erzwungen werden (Menü → Debug).

**Bedingter
Breakpoint
z. B. „c < 100“**



The screenshot shows the MATLAB Editor window with the file `debugging.m` open. The script contains the following code:

```
1 a = 23;  
2 b = 4;  
3  
4 c = a * b;  
5 d = c * b;  
6 c = d^2;  
7  
8 for index = 1 : c  
9  
10     b = b + 1;  
11  
12 end  
13  
14 e = fakultaet(a);  
15  
16 disp(b)  
17 disp(e)
```

A conditional breakpoint is set on line 5, indicated by a yellow circle with a green arrow. An arrow from the text "Bedingter Breakpoint z. B. „c < 100“" points to this breakpoint. The status bar at the bottom shows "Ln 5 Col 1 OVR".

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Umleitung von Befehlen auf die DOS-Ebene:

- Ein '!' vor einem Befehl bewirkt, dass dieser in der DOS-Kommandozeile ausgeführt und die Ausgaben nach Matlab umgeleitet werden.
- Bsp.:

```
>> !ipconfig
```

```
Windows-IP-Konfiguration
```

```
Drahtlos-LAN-Adapter Drahtlosnetzwerkverbindung 2:
```

```
Medienstatus. . . . . : Medium getrennt  
Verbindungsspezifisches DNS-Suffix:
```

```
Mobiler Breitbandadapter Mobile Breitbandverbindung:
```

```
Medienstatus. . . . . : Medium getrennt  
Verbindungsspezifisches DNS-Suffix:
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Der Befehl

```
>> dos('ipconfig');
```

bewirkt die gleiche Reaktion, während bei Zuweisung einer Variablen:

```
>> [s, w] = dos('ipconfig');
```

```
>> s
```

```
s =
```

```
0
```

```
>> w
```

```
w =
```

```
Windows-IP-Konfiguration ....
```

Das `s` den FehlerCode und `w` das “Echo” enthält.

```
>> [s, w] = dos('ipconfig', '-echo');
```

würde wieder die Bildschirmausgabe erzwingen.

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Function-Handle:

Bei Übergeben von Funktionen an Funktionen, ist es sinnvoller mit sogenannten Function-Handles zu arbeiten (wird auch bei eingebauten Matlab-Funktionen verwendet). Function-Handle arbeiten wie folgt

```
>> MyFunHandle = @myfun;  
>> feval(MyFunHandle,2,3)
```

→ Erzeugen des Handle
→ Auswerten der Funktion mit 1 u. 2

```
ans =  
     6
```

Es Funktioniert auch:

```
>> MyFunHandle = str2func('myfun');
```

Umkehrbefehl:

```
>> fstring = func2str(MyFunHandle)
```

```
fstring =  
      myfun
```



5. M-Programmierung – 5.3 Hilfreiche Funktionen

Anwendung:

Funktion 1:

```
function output = myfun(input)

    output = sin(input);

end
```

Funktion 2:

```
function vector = make_vector(fhandle)

    x = linspace(0,1,100)
    vector = feval(fhandle,x);

end
```

```
>> MyFunHandle = @myfun;
>> make_vector(MyFunHandle)
```

```
x = ...
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

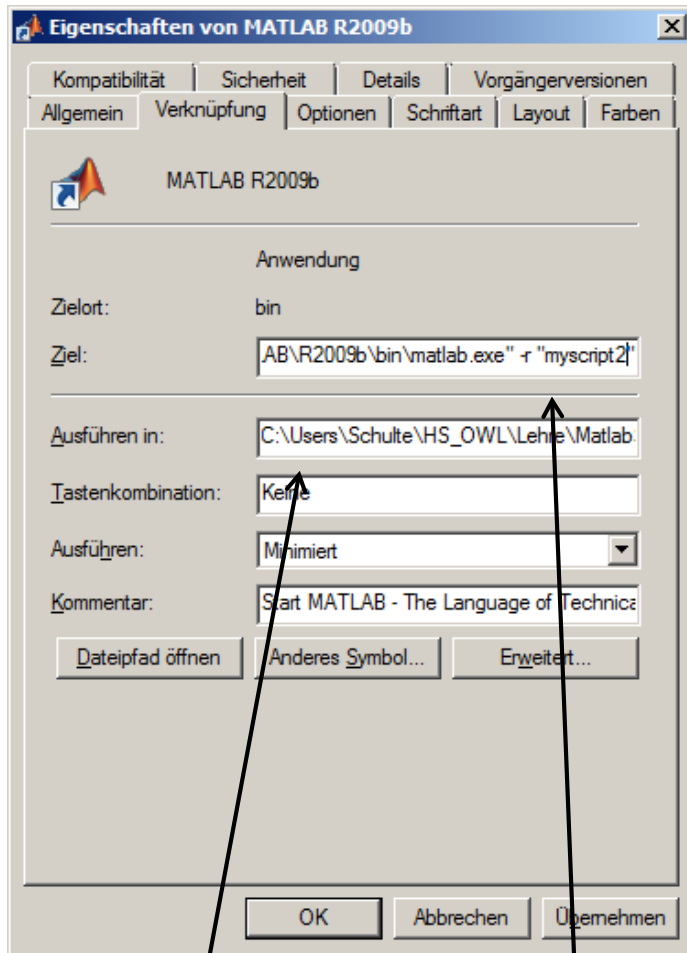
Initialisierungen beim Starten von Matlab:

- Während des Starts und nach Initialisierung der wichtigsten Funktionen ruft Matlab das m-Skript `matlabrc` auf. (`<matlab_root>/toolbox/local/matlabrc.m`)
- `matlabrc` ruft wiederum das Skript `startup` im Startverzeichnis auf.
- Das Startverhalten kann außerdem bei Links oder bei Aufrufen durch eine Batch-Datei durch Startup-Optionen beeinflusst werden:

Beispiel:

```
"MATLAB_ROOT\bin\win32\MATLAB.exe" -r mymfunction;exit;
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen



Startverzeichnis

Startup-Optionen

Startup-Optionen:

-c licensefile

-h or -help

→ Anzeigen Startup-Optionen

-logfile "logfile"

→ Automatisches Log-File

-minimize

→ MATLAB startet minimiert.

-nojvm

→ Startet MATLAB ohne
Microsystems JVM™
(keine Grafikfunktionen)

-r "statement"

→ Automatischer Start von
statement (M-Anweisung)

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Zugehörige Funktionen:

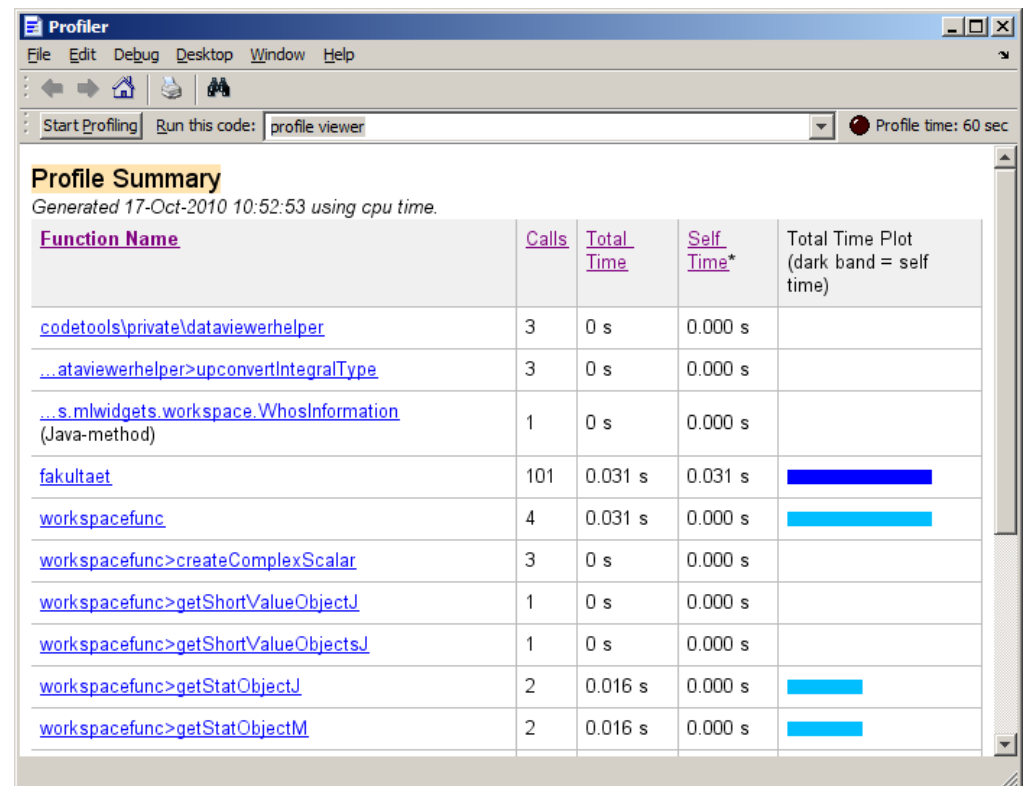
- `char(g)` → Rückkonvertierung zum String.
- `formula(fg)` → Identisch zu `char(g)`.
- `argnames(fun)` → Angabe der Argumente als Cell-Array von Strings.
- `vectorize(fun)` → Vektorisierung (wirkt wie das Einfügen von `(.)` vor den multiplikativen Operatoren).

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Matlab-Profiler:

Der Matlab-Profiler ermöglicht das Zeitverhalten und die Aufrufe einer Matlab-Anweisung und der Unteraufrufe zu protokollieren:

```
>> profile on
>> fakultaet(100);
>> profile viewer
```



5. M-Programmierung – 5.3 Hilfreiche Funktionen

Zeitmessung in Skripten/Funktionen:

```
tic;  
<Anweisung>  
<Anweisung>  
<Anweisung>  
..  
<Anweisung>  
T = toc;
```

>> T → Zeit zwischen `tic` und `toc` in Sekunden

```
T =  
    0.0036
```

Alternativ:

```
t1 = tic;  
...  
T = toc(t1);
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Weitere Zeitfunktionen:

<code>cputime</code>	→ Prozessorzeit in Sekunden
<code>clock</code>	→ Numerisches Array mit Datum und Urzeit
<code>date</code>	→ Datumstring
<code>datestr</code>	→ Konvertierung zum Datumstring

Bsp.:

```
>> date
ans =
18-Oct-2010

>> clock
ans =
1.0e+003 *

    2.0100    0.0100    0.0180    0.0130    0.0430    0.0480

>> datestr(clock)
ans =
18-Oct-2010 13:44:18
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Aufruf-Struktur:

Die `depfun`-Anweisung ermöglicht die Ermittlung der abhängigen Funktionen:

```
>> depfun('mscript11','-all')
=====
depfun report summary:
-----
-> trace list:           9 files    (total)
                        1 files    (total arguments)
                        0 files    (arguments off MATLABPATH)
                        0 files    (argument duplicates on
MATLABPATH)
-> builtin list:         31 names
-> MATLAB classes:      1 names    (builtin, MATLAB OOPS)
-> problem list:        0 files    (argument)
                        0 files    (other)
-> problem symbols:     NOT IMPLEMENTED
-> eval strings:        0 files    (calling eval, etc.)
-> called from list:    1 files    (argument unreferenced)
                        0 files    (argument referenced) ...
```


5. M-Programmierung – 5.3 Hilfreiche Funktionen

Speicherbelegung:

Die `memory`-Anweisung ermöglicht die Ermittlung des Speicherzustand:

```
>> memory
Maximum possible array:          6112 MB (6.409e+009 bytes) *
Memory available for all arrays:  6112 MB (6.409e+009 bytes) *
Memory used by MATLAB:           587 MB (6.154e+008 bytes)
Physical Memory (RAM):           3956 MB (4.148e+009 bytes)
```

* Limited by System Memory (physical + swap file) available.

```
>> [USERVIEW, SYSTEMVIEW] = memory
```

USERVIEW =

```
MaxPossibleArrayBytes: 6.4088e+009
MemAvailableAllArrays: 6.4088e+009
MemUsedMATLAB: 615305216
```

SYSTEMVIEW =

```
VirtualAddressSpace: [1x1 struct]
SystemMemory: [1x1 struct]
PhysicalMemory: [1x1 struct]
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

Die `pack`-Anweisung konsolidiert den Speicher und gibt unbenutzte Reserven frei (ähnlich einer Defragmentierung)

```
>> pack
```

5. M-Programmierung – 5.3 Hilfreiche Funktionen

P-Code-Funktion:

Matlab bietet die Möglichkeit m-Funktionen in sog. p-kodierte Funktionen (P-Code-Funktion) umzuwandeln.

- P-Code-Funktion sind nicht mehr als Quellcode lesbar und
- werden schneller „etwas“ schneller ausgeführt.
- Bei gleichnamigen m- und p-Funktionen im Suchpfad hat die p-Funktion Vorrang.
- Die `help`-Funktionen greift aber weiter auf den Header-Text der m-Funktion zu. (Häufig werden ansonsten „leere“ m-Dateien mit dem Hilfetext verwendet.)

```
pcode <dateiname>
```

→ wandelt m-Funktionen in P-Code-Funktion um

Bsp.:

```
>> pcode myfun
```

